

Notebook: all_algo_comparisons/naive_bayes_comparison.ipynb

[--- MARKDOWN CELL ---]

Naive Bayes - Algorithm Comparison

Comparing Gaussian Naive Bayes from-scratch vs sklearn on Breast Cancer and Wine datasets.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer, load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB as SklearnGNB
from sklearn.metrics import accuracy_score, confusion_matrix
np.random.seed(42)
```

[--- CODE CELL ---]

```
class GaussianNBScratch:
    """Gaussian Naive Bayes from scratch."""
    def __init__(self):
        self.classes = None
        self.priors = None
        self.means = None
        self.variances = None
    def fit(self, X, y):
        X, y = np.asarray(X), np.asarray(y)
        if X.ndim == 1:
            X = X.reshape(-1, 1)
        self.classes, counts = np.unique(y, return_counts=True)
        self.priors = counts / counts.sum()
        n_classes, n_features = len(self.classes), X.shape[1]
        self.means = np.zeros((n_classes, n_features))
        self.variances = np.zeros((n_classes, n_features))
        for i, cls in enumerate(self.classes):
            X_c = X[y == cls]
            self.means[i] = X_c.mean(axis=0)
            self.variances[i] = X_c.var(axis=0) + 1e-6
        return self
    def _log_gaussian(self, x, mean, var):
        coeff = -0.5 * np.log(2 * np.pi * var)
        exponent = -((x - mean) ** 2) / (2 * var)
        return np.sum(coeff + exponent)
    def predict(self, X):
        X = np.asarray(X)
        if X.ndim == 1:
            X = X.reshape(-1, 1)
        preds = []
        for x in X:
            log_posts = []
            for i in range(len(self.classes)):
                log_lik = self._log_gaussian(x, self.means[i], self.variances[i])
                log_posts.append(log_lik + np.log(self.priors[i]))
            preds.append(self.classes[np.argmax(log_posts)])
        return np.array(preds)
    def score(self, X, y):
        return accuracy_score(np.asarray(y), self.predict(X))
```

[--- CODE CELL ---]

```
# Breast Cancer
bc = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(bc.data, bc.target, test_size=0.2, random_state=42)
nb_scratch = GaussianNBScratch().fit(X_train, y_train)
nb_sklearn = SklearnGNB().fit(X_train, y_train)
acc_scratch_bc = nb_scratch.score(X_test, y_test)
acc_sklearn_bc = nb_sklearn.score(X_test, y_test)
print(f"Breast Cancer - Scratch: {acc_scratch_bc:.4f}, Sklearn: {acc_sklearn_bc:.4f}")
```

[--- CODE CELL ---]

```
# Wine
wine = load_wine()
X_train, X_test, y_train, y_test = train_test_split(wine.data, wine.target, test_size=0.2, random_state=42)
nb_scratch = GaussianNBScratch().fit(X_train, y_train)
nb_sklearn = SklearnGNB().fit(X_train, y_train)
acc_scratch_wine = nb_scratch.score(X_test, y_test)
acc_sklearn_wine = nb_sklearn.score(X_test, y_test)
print(f"Wine - Scratch: {acc_scratch_wine:.4f}, Sklearn: {acc_sklearn_wine:.4f}")
```

[--- CODE CELL ---]

```
# Comparison plot
datasets = ['Breast Cancer', 'Wine']
scratch = [acc_scratch_bc, acc_scratch_wine]
sklearn = [acc_sklearn_bc, acc_sklearn_wine]

x = np.arange(2)
plt.figure(figsize=(10, 6))
bars1 = plt.bar(x - 0.2, scratch, 0.4, label='From Scratch', color='steelblue')
bars2 = plt.bar(x + 0.2, sklearn, 0.4, label='Sklearn', color='coral')

plt.xticks(x, datasets)
plt.ylabel('Accuracy')
plt.title('Naive Bayes: From Scratch vs Sklearn Comparison')
plt.legend()
plt.ylim(0.8, 1.05)
plt.grid(True, alpha=0.3, axis='y')

for bar in bars1:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{bar.get_height():.4f}', ha='center', va='bottom')
for bar in bars2:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{bar.get_height():.4f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()
```

[--- CODE CELL ---]

```
# Summary
print("\nNaive Bayes Comparison Summary:")
print("="*50)
print(f"{'Dataset':<20} {'From Scratch':<15} {'Sklearn':<15}")
print("-"*50)
print(f"{'Breast Cancer':<20} {acc_scratch_bc:<15.4f} {acc_sklearn_bc:<15.4f}")
print(f"{'Wine':<20} {acc_scratch_wine:<15.4f} {acc_sklearn_wine:<15.4f}")
```